

OrgDesk

A configurable internal operations management platform for organizations.

PRODUCT REQUIREMENT DATA

Phase 0 — Product Definition

0.1 Product concept

Our platform is a **multi-organization internal operations management system**.

Each organization can configure its own:

- Departments
- User types
- Request categories
- Members

The platform provides the common functionality:

- Authentication
- Organization management
- Request management
- Status tracking
- Communication
- Notifications
- Dashboard
- History

0.2 Core workflow

Organization Admin



Creates Organization



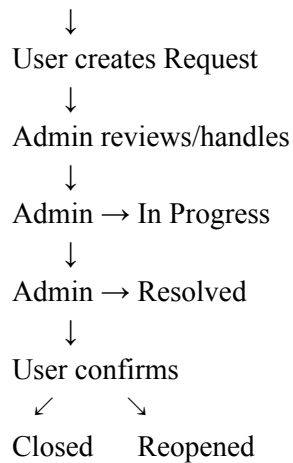
Configures Organization



Invites Users



User Registration



0.3 Our MVP boundary

MUST HAVE

- Organization creation
- Admin account
- Organization configuration
- User invitations
- User registration
- Login
- Admin/User authorization
- Request creation
- Request categories
- Priority
- Status workflow
- Comments
- Request history
- Dashboard
- Search/filter/pagination
- Organization data isolation

SHOULD HAVE

- Notifications
- Attachments
- User feedback
- Basic analytics

FUTURE

- Email service
- Advanced permissions
- Multiple admin levels
- SLA management
- Advanced reporting

0.4 Problem Statement

Organizations often handle internal requests through **WhatsApp, email, phone calls, spreadsheets, or paper**.

This creates problems such as:

- Requests getting lost
- No clear status tracking
- Difficult to know which requests are pending
- Poor communication between users and administration
- No proper request history
- Difficult to measure operational workload

Our platform provides **one centralized system** to create, manage, track, and resolve internal requests.

0.5 Target Organizations

The same platform should support:

- Schools
- Colleges
- Companies
- Hospitals
- NGOs
- Hostels
- Other organizations

The organization configures the platform according to its own needs.

0.6 Main Target Users

Organization Admin

Manages the organization and handles requests.

Organization User

Raises requests and tracks their progress.

0.7 Product Goal

Provide a configurable and secure platform that allows different organizations to manage their internal requests and operational issues in one centralized system.

Phase 1 — Functional Requirements

Functional requirements describe **what the system must do**.

We'll keep them organized by module.

1. Organization

The system must allow an Admin to:

- Create an organization
- View/update organization information
- Configure departments
- Configure user types
- Configure request categories
- Invite users

2. User

The system must allow users to:

- Register through an invitation
- Login/logout
- View their profile
- View their organization information
- View their own requests

3. Request

Users must be able to:

- Create requests
- Select category
- Set priority
- Add title and description
- View request status
- Add comments

Admins must be able to:

- View organization requests
- Filter/search requests
- Change priority
- Change status
- Add comments
- Resolve requests

4. Request lifecycle

The system must support:

OPEN → IN_PROGRESS → RESOLVED → CLOSED

and:

RESOLVED → REOPENED → IN_PROGRESS

5. Security

The system must:

- Authenticate users
- Authorize Admin/User actions
- Isolate organization data
- Prevent unauthorized request access

Phase 1 — Non-Functional Requirements

These describe **how well the system should work**, rather than what it does.

For our project, we only need practical requirements.

1. Security

- Passwords must never be stored as plain text.
- APIs must require authentication where necessary.
- Users can access only data belonging to their organization.
- Admin-only operations must be protected.
- JWT tokens should be handled securely.

2. Performance

The application should respond quickly for normal operations.

We should support:

- Pagination for large request lists
- Database indexes where useful
- Efficient queries

We don't need to target millions of users.

3. Reliability

- Proper exception handling
- Input validation
- Consistent API error responses
- Database constraints

4. Usability

- Simple interface
- Responsive design
- Clear status indicators
- Easy request creation

5. Maintainability

The code should have:

- Clear separation of responsibilities
- Meaningful naming
- Consistent project structure
- Environment-based configuration
- Documentation
- Tests for important business logic

6. Scalability

The architecture should allow the application to grow later without requiring a complete rewrite.

Phase 1 — Roles & Permissions

We'll keep only **two roles** for MVP.

1. ADMIN

The Admin manages the organization and its requests.

Action	Admin
Manage organization	✓
Manage departments	✓
Manage user types	✓
Manage categories	✓
Invite users	✓
View all requests	✓
Create request	✓
Change request status	✓
Change priority	✓

Add comments	✓
View request history	✓
Manage users	✓

2. USER

Action	User
View organization	✓
Update own profile	✓
Create request	✓
View own requests	✓
Add comments	✓
View request history	✓
Confirm resolution	✓
Reopen request	✓
Manage organization	✗
View other users' requests	✗
Manage categories	✗
Invite users	✗

Phase 1 — Main Use Cases

A **use case** describes something a user wants to accomplish with the system.

We only need the important ones now.

Admin Use Cases

Organization

1. Create organization
2. Update organization

3. Configure departments
4. Configure user types
5. Configure request categories

Users

6. Invite user
7. View users
8. Manage users

Requests

9. View requests
10. Filter/search requests
11. Review request
12. Change priority
13. Change status
14. Add comment
15. Resolve request

Dashboard

16. View request statistics
-

User Use Cases

Account

1. Accept invitation
2. Complete registration
3. Login
4. Manage profile

Requests

5. Create request
 6. View own requests
 7. View request details
 8. Add comment
 9. Track status
 10. Confirm resolution
 11. Reopen request
-

Our core use-case chain

Admin

↓

Create Organization

↓

Configure Organization



Invite User



User Registers



User Creates Request



Admin Handles Request



User Confirms Resolution

Admin stories

- As an Admin, I want to create an organization so that I can manage it through the platform.
- As an Admin, I want to configure departments so that my organization structure is represented.
- As an Admin, I want to configure user types so that members can be categorized.
- As an Admin, I want to configure request categories so that users can submit relevant requests.
- As an Admin, I want to invite users so that only authorized members can join.
- As an Admin, I want to manage requests so that organizational issues can be resolved.
- As an Admin, I want to view request statistics so that I can understand operational workload.

User stories

- As a User, I want to accept an invitation so that I can join my organization.
- As a User, I want to create an account so that I can access the platform.
- As a User, I want to submit a request so that I can report an issue or ask for help.
- As a User, I want to track my request so that I know its current status.
- As a User, I want to communicate with the Admin through comments.
- As a User, I want to confirm or reopen a resolved request.

Business Rules

Business rules are important because they define **what the system is allowed to do**.

We'll keep them simple.

Organization rules

1. An organization must have at least one Admin.
2. Every user belongs to one organization.
3. A user cannot access another organization's data.
4. Only the Admin can configure the organization.

Invitation rules

5. An invitation belongs to one organization.
6. An invitation can be used only once.
7. An expired invitation cannot be used.
8. An invitation determines the user's organization and initial User Type/Department.

Request rules

9. Every request belongs to exactly one organization.
10. Every request has a requester.
11. A User can view only their own requests.
12. An Admin can view requests belonging to their organization.
13. Only an Admin can move a request into **IN_PROGRESS** or **RESOLVED**.
14. A User can close a resolved request after confirming the resolution.
15. A User can reopen a resolved request if the issue is not solved.

Data rules

16. Required information cannot be empty.
17. Deleted configuration should not break existing requests.
18. Important request changes should be recorded in history.

These rules will later directly influence our **database design, API authorization, and service-layer business logic**.